DeepTrustxAI Cloud Security Workshop - Revised Modules

DeepTrustxAI

Cloud Security Workshop

Revised module-wise structure

CCSP concepts first · Vulnerable deployment second · Scanner usage third · Open-source tools fourth

---

Corrected Workshop Flow

Module	Focus	Outcome
Module 1	CCSP and cloud vulnerability concepts	Participants understand what to look for before touching tools
Module 2	Vulnerable app and deployment	Participants see how weaknesses are intentionally introduced in a lab
Module 3	Scanner and CloudGuard usage	Participants run scans, read results and map evidence
Module 4	Open-source tools and usage	Participants understand tool coverage and practical usage

> Teach the concept, show the vulnerable deployment, scan it, then explain the tools.

Two-Day Delivery Plan

Day	Session	Duration	Main output
Day 1	Module 1: CCSP and vulnerability concepts	3.5 hrs	Concept map and CCSP domain mapping
Day 1	Module 2: Vulnerable app/deployment, part 1	2 hrs	Lab architecture and expected findings
Day 2	Module 2: Vulnerable app/deployment, part 2	1.5 hrs	Deployed/loaded vulnerable lab
Day 2	Module 3: Scanner and CloudGuard usage	3 hrs	Scan output, evidence and GRC mapping
Day 2	Module 4 + capstone	2.5 hrs	Tool map and executive risk story

---

1

Module 1

CCSP and cloud vulnerability concepts

Build the security vocabulary before introducing scanners.

Module 1: CCSP Lens

CCSP Domain	What we teach in this workshop
D1 Cloud Concepts, Architecture and Design	Models, shared responsibility, cloud planes and secure design
D2 Cloud Data Security	Storage, encryption, backup, retention and data exposure
D3 Cloud Platform and Infrastructure Security	IAM, network, compute, Kubernetes and cloud infrastructure
D4 Cloud Application Security	APIs, CI/CD, vulnerable dependencies, secrets and serverless
D5 Cloud Security Operations	Logging, monitoring, scanning, incident response and evidence

CCSP Domain	What we teach in this workshop
D6 Legal, Risk and Compliance	Control mapping, owners, audit proof and GRC

Cloud Vulnerability Concepts

Area	Typical weakness	Impact
Identity	Wildcard roles, stale keys, missing MFA	Privilege escalation
Network	Public admin ports, weak segmentation	Direct exposure
Storage	Public buckets, exposed backups	Data leakage
Compute	Vulnerable image, metadata abuse	Workload compromise
IaC	Unsafe defaults and drift	Repeatable insecure deployment
App/API	SSRF, injection, weak auth, dependencies	App-to-cloud attack path
Logs/GRC	Missing evidence and ownership	Unclosed risk

How Vulnerabilities Creep In

DesignWrong trust boundary or missing owner

BuildUnsafe IaC, dependency or secret

DeployBroad role, public rule or weak default

OperateManual drift, stale asset or missing log

CloseNo proof, no owner or no rescan

Message: A finding is not just a technical issue. It needs evidence, ownership and closure proof.

Module 1 Exercise

Participants classify sample findings:

1. Which CCSP domain does it belong to?
2. Which control failed?
3. What evidence proves it?
4. Who owns remediation?
5. Is it design, deployment or operational drift?

Output: CCSP mapping table and vulnerability concept map.

2

Module 2

Vulnerable application and deployment

Show the target before running the scanner.

Vulnerable Lab Architecture

Vulnerable appWeak endpoint, dependency or header

Cloud resourcePublic storage, open ingress or broad IAM

IaC sourceUnsafe Terraform/YAML sample

WorkerScanner runs separately from portal

GRCEvidence, control and owner

Lab rule: create temporary vulnerable resources inside isolated sandbox accounts/projects/namespaces, not full cloud accounts.

What The Lab Should Contain

Component	Intentional weakness
Web/API app	Vulnerable dependency, weak headers, test endpoint or insecure config
Config/source	Dummy hardcoded secret and unsafe sample file

Component	Intentional weakness
Storage	Public policy or disabled public access block
IAM/service account	Over-permissive policy or wildcard action
Network	Public admin ingress or overly broad firewall rule
Kubernetes	Privileged pod, wildcard RBAC, NodePort or missing NetworkPolicy
Logging	Missing audit trail or weak evidence retention

Lab Safety Guardrails

- Use only approved sandbox accounts, projects or namespaces.
- Use TTL tags and automatic cleanup.
- Never put real secrets in a vulnerable fixture.
- Keep budget alerts enabled.
- Prefer IaC/local vulnerable files when live cloud credentials are unavailable.
- Keep pre-recorded scanner output as fallback.

Expected output: lab ID, tenant ID, expected findings checklist and cleanup proof.

Module 2 Exercise

Before scanning, participants must predict:

Weakness	Expected scanner/tool	Evidence expected
Public bucket	Cloud/IaC scanner	Policy and asset reference
Wildcard IAM	Cloud scanner/CloudFox	Policy finding and principal
Vulnerable dependency	Trivy/Safety	Package and CVE
Hardcoded secret	Gitleaks	Redacted secret finding
Web exposure	Nuclei/ZAP	URL, template and proof

3

Module 3

Scanner and CloudGuard usage

Run the scanner, understand the output, and convert findings into evidence.

CloudGuard Scanner Workflow

PortalScope, tenant and credential reference

QueuePostgreSQL scan jobs and status

WorkersAWS, GCP, Kubernetes, IaC, web, secrets and dependencies

EvidenceNormalized artifacts and checksums

GRCControl, owner and remediation proof

Key point: the portal is decoupled from scanner jobs.

How To Use The Scanner

1. Open CloudGuard portal.
2. Add/select credentials in Manage Credentials.
3. Choose AWS, GCP, Kubernetes, IaC or web/app target.
4. Start scan or launch a sandbox lab.
5. Watch job status: queued, running, completed, failed or dead-letter.
6. Review findings by severity, asset and source tool.
7. Validate important findings and reduce false positives.
8. Map evidence to GRC controls.
9. Assign owner, remediation and closure proof.

What The Scanner Covers

Scanner stream	Coverage
AWS/GCP	IAM, storage, network, exposed services and cloud posture
Kubernetes	Namespaces, pods, services, RBAC and network-policy posture
IaC	Terraform, CloudFormation, Kubernetes YAML and JSON

Scanner stream	Coverage
Web/App	HTTP exposure and common web weaknesses
Secrets	Hardcoded credential patterns
Dependencies	Vulnerable packages and containers
Evidence/GRC	Connector payloads, checksums, controls and audit proof

---

Findings To GRC Evidence

Scanner output	GRC output
Finding ID	Control evidence reference
Asset/resource	Asset owner and business context
Severity	Risk priority after context
Proof payload	Validation job
Recommended fix	Remediation action
Rescan/cleanup proof	Closure evidence

Message: the scanner is useful only when its output becomes a decision and closure workflow.

Module 3 Demo Script

1. Upload vulnerable IaC or launch sandbox lab.
2. Start authorized scan.
3. Show queue and worker status.
4. Open finding details.
5. Compare actual findings with expected findings.
6. Submit or view evidence artifact.
7. Map finding to GRC control.
8. Create validation/remediation record.

4

Module 4

Open-source tools and their usage

Explain what powers the scanner and how teams can use tools independently.

Open-Source Tool Map

Category	Tools	Use
IaC scanning	Checkov, KICS, Terrascan, tfsec	Terraform/YAML policy checks
Cloud posture	Prowler, ScoutSuite, CloudFox	Cloud posture and identity attack paths
Container/dependency	Trivy, Grype, Safety	Images, packages and libraries
Secret scanning	Gitleaks, TruffleHog	Hardcoded secrets and tokens
Web testing	OWASP ZAP, Nuclei	Web exposure and known checks
Kubernetes	Kube-bench, Kubescape, Polaris, Trivy config	Cluster and manifest posture
Code review	Semgrep, Bandit	Insecure code patterns

Tool Usage Rules

- Run only on authorized assets.
- Pin tool versions for reproducibility.
- Store raw output as evidence, but redact secrets.
- Normalize outputs before sending to GRC.
- Deduplicate findings across tools.
- Prioritize by exposure, identity impact and data sensitivity.
- Track false positives and accepted risk separately.

Module 4 Exercise

Given four tool outputs:

1. Identify which finding is highest risk.

2. Explain whether CloudGuard detected it directly or through a connector.
3. Normalize one output into an evidence payload.
4. Map it to a CCSP domain and GRC control.
5. Define remediation proof.

Capstone: Full Story

Teams present the full chain:

Step	What to explain
Concept	CCSP domain and vulnerability class
Vulnerable deployment	How the weakness was introduced
Scanner	Which scan/tool found it
Evidence	What proof was stored
GRC	Which control, owner and due date
Closure	How remediation will be validated

What To Tell Sponsors

The workshop now follows a clean learning path:

1. First, participants learn CCSP and cloud vulnerability concepts.
2. Then they see how a vulnerable app/deployment is built safely.
3. Then they use CloudGuard to scan, validate and map evidence.
4. Finally, they learn the open-source tools behind the scanner coverage.

This makes the session concept-first, demo-backed and evidence-driven.

DeepTrustxAI

Concept → Vulnerable Deployment → Scanner → Tools

Revised for module-wise delivery.